

Real-Time Traffic Light Control System Using IPC

Project #2 — Design Report

Birzeit University — Faculty of Engineering and Technology

Electrical & Computer Engineering Department

2nd semester 2025 / 2026

Project Scope	Multi-process simulation of a four-way intersection with realistic traffic-light control, pedestrian crossings, emergency-vehicle priority, and a 3-D OpenGL visualisation.
Languages & Tools	C99, POSIX / System V IPC, gcc, GNU Make, GDB, freeglut + OpenGL
IPC Mechanisms	Shared memory (state), System V semaphores (mutual exclusion), System V message queues (events & commands)
Processes	9 cooperating processes: main, controller, logger, 4 × traffic light, vehicle detector, pedestrian generator, emergency generator, OpenGL GUI

1. Introduction

This report explains the design of a real-time traffic-light control system implemented for ENCS4330 Project #2. The objective is to model a busy four-way intersection in which vehicles, pedestrians and emergency vehicles all interact with traffic signals safely and predictably. The system has to satisfy real-time deadlines (a yellow light must last for a specific number of seconds, a pedestrian request must be served within a maximum waiting time, an emergency vehicle must be cleared within a few seconds, and so on) while at the same time guaranteeing safety invariants such as *two conflicting directions are never green simultaneously*.

The natural way to model such a system on Linux is as a collection of cooperating processes that exchange information through inter-process communication (IPC). This project uses three of the classic System V IPC mechanisms taught in class — shared memory, semaphores, and message queues — and combines them with the OpenGL (freeglut) library for visualisation. The remainder of this report documents *which* mechanism was chosen for *which* kind of data, and explains *why* alternatives were rejected.

1.1 High-level requirements

- Four directions (North, South, East, West), each with red, yellow, and green states.
- A controller that runs a phase state machine and enforces safety invariants.
- Random arrival of vehicles, pedestrians, and emergency vehicles.
- Real-time logging of every system event with timestamps.
- Tunable runtime parameters read from a plain-text file.
- Realistic OpenGL visualisation with interactive keyboard input.

2. System Architecture

The system is decomposed into nine cooperating processes. The responsibility of each is summarised in Table 1. The launcher (main) is responsible for creating every IPC resource, parsing the configuration file, then forking the remaining eight processes and supervising them. When the user presses Ctrl-C (or *q* in the GUI), main sends SIGTERM to its children and finally destroys the IPC objects so the kernel is left clean.

Process	Role	Key data it produces / consumes
main	Boots the system, owns all IPC resources, supervises children, performs graceful shutdown.	Creates SHM, semaphores, queues. Parses configuration.
controller	The brain. Runs the phase state machine, enforces safety, computes adaptive green durations, handles emergencies.	Reads EVT queue; writes commands to CMD queue; updates SHM.

Process	Role	Key data it produces / consumes
traffic_light (× 4)	Driver for one physical light. Receives commands, simulates the lamp, sends acks, reports faults.	Reads CMD queue (filtered by mtype = base + direction). Writes SHM light colour, sends ACK or FAULT events.
vehicle_detector	Inductive-loop / camera simulator. Pushes vehicle-arrival events at a configurable rate per direction.	Writes to EVT queue (MTYPE_EVT_VEHICLE).
pedestrian	Pedestrian-button simulator. Generates crossing requests with a configurable probability per direction.	Writes to EVT queue (MTYPE_EVT_PEDESTRIAN).
emergency	Rare emergency-vehicle generator with a per-mille rate.	Writes to EVT queue (MTYPE_EVT_EMERGENCY).
logger	Drains the log queue to disk and stdout. Isolates I/O latency from the real-time loops.	Reads LOG queue, writes logs/system.log.
gui_opengl	3-D visualisation. Renders the intersection, cars, pedestrians, traffic lights, and the on-screen HUD. Also lets the user inject events with the keyboard.	Reads SHM; can write to EVT queue when keys are pressed.

Table 1 — Process responsibilities

2.1 Communication topology

Three message queues are used. Splitting the queues by direction of information flow keeps the design clean and lets each consumer filter by mtype efficiently:

- **CMD queue** — controller → traffic lights (and acks back). Each direction reads only messages whose mtype = MTYPE_CMD_BASE + direction.
- **EVT queue** — sensors and the GUI → controller. Different mtotypes identify vehicle, pedestrian, emergency, and fault events.
- **LOG queue** — every process → logger. The producer side always uses IPC_NOWAIT so it never blocks on the logger.

2.2 Shared blackboard

All real-time state lives in a single shared-memory segment, the *blackboard*. The structure is defined once in common.h and is identical for every process. Every read or write that spans more than one field is wrapped in sem_lock(semid) / sem_unlock(semid), which is a binary semaphore providing mutual exclusion. The blackboard contains:

- Light colour and last-change time for each direction.
- Vehicle queue length per direction.

- Pedestrian-request flags and their request-time stamps.
- Emergency flag and direction.
- Current phase, phase-started time, and remaining countdown.
- All tunable parameters loaded from the configuration file.
- Aggregate counters: vehicles served, safety violations, etc.

3. Why these IPC choices (and not others)

The single most important design decision in this project is the split of responsibilities between shared memory and message queues. Both are powerful, both are taught in class, and using them in the wrong place can lead to either lost events or stale state. The choices made here are explained below.

3.1 Why shared memory for the blackboard

The OpenGL GUI redraws the scene roughly sixteen times a second; the controller checks queue lengths several times a second; every traffic-light process reads its own colour after every command. If all of this state were transmitted by messages, every consumer would have to subscribe to a constant flow of "state update" broadcasts. Shared memory is fundamentally *broadcast by construction*: any process attaches once and then reads at native memory speed, without context switches into the kernel. The trade-off is that we now need explicit mutual exclusion (see §3.2).

An obvious alternative is to keep state inside the controller and let everyone ask for it via a message queue. This would work but adds latency on every read, doubles the message traffic, and makes the controller a single point of failure. Pipes or sockets would suffer from the same problem and would also require manual framing of every record.

3.2 Why a semaphore for mutual exclusion

The blackboard contains many related fields that must be updated *atomically* — for example, when the controller increases a vehicle queue length it must also update the corresponding `last_change` timestamp. A System V semaphore initialised to 1 acts as a binary mutex: `P()` at the start of a critical section, `V()` at the end. We chose System V over POSIX `sem_t` because the latter is thread-oriented and trickier to share between unrelated processes without putting the semaphore in shared memory itself.

The semaphore is created with the `SEM_UNDO` flag in every operation, so that if a process crashes while holding the lock, the kernel automatically releases it. This protects the whole system from the classic deadlock-after-crash failure mode.

3.3 Why message queues for events and commands

Events are *discrete*: a vehicle either arrives or it doesn't. If we tried to carry them in shared memory we would need a circular buffer plus our own semaphore-based producer-consumer protocol — exactly what message queues already give us, with the added benefit of `mtype` filtering so each traffic-light process can grab only its own commands. `IPC_NOWAIT` on the producer side and a small `usleep()` in the consumer loops prevents both lost messages and busy-waiting.

An alternative would have been UNIX-domain sockets or POSIX message queues (`mq_open`). They have richer feature sets but require `fcntl`, more setup code, and we do not need any of their extras (priorities, notifications, etc.) for this scale of project. Named pipes were rejected because they don't support `mtype` filtering, which is essential to let each traffic-light process receive only the commands addressed to it.

3.4 Trade-off comparison table

Kind of data	Chosen mechanism	Reason / rejected alternatives
Global state	Shared memory (+ semaphore mutex)	Read very often, by many processes; messages would be wasteful. Plain pipes lack random access; POSIX shm needs mapping in every process.
Discrete events	System V message queues	FIFO ordering, never lost, mtype filtering; pipes need manual framing; POSIX mqueue needs librt and isn't necessary here.
Mutual exclusion	System V semaphore + SEM_UNDO	Survives process crashes; POSIX sem_t needs shared memory placement; pthread mutex requires PTHREAD_PROCESS_SHARED setup.
Logging	Dedicated message queue, IPC_NOWAIT	Producers never block on disk; a shared file would race; a syslog dependency would add an external service.

Table 2 — IPC choice summary

4. Phase State Machine

The controller is built around a finite-state machine whose states are *traffic phases*. The legal phases and the transitions between them are listed below.

4.1 Phases

Phase	Meaning	Default duration
NS-GREEN	North & South go; East & West stop.	8–15 s (adaptive)
NS-YELLOW	Clearing the N/S directions.	3 s
ALL-RED-1	Safety clearance interval.	2 s
EW-GREEN	East & West go; North & South stop.	8–15 s (adaptive)
EW-YELLOW	Clearing the E/W directions.	3 s
ALL-RED-2	Safety clearance interval.	2 s
PEDESTRIAN	All vehicle lights red; crosswalk active.	10 s
EMERGENCY	One direction green, others red.	$\geq T_GREEN_MIN$

4.2 Transition rules and safety invariants

- After NS-GREEN comes NS-YELLOW, then ALL-RED. *Yellow is never skipped.*
- Pedestrian phase can only begin from ALL-RED (every vehicle light is red).
- Emergency mode safely transitions any current green to yellow, then red, before granting the emergency direction its green.
- After every transition the controller calls `state_is_safe()`, which checks that NS-green and EW-green are not both true.

4.3 Adaptive green duration

Modern signal controllers vary the green duration based on demand. Our controller does this with a small formula:

```
duration = clamp(T_GREEN_MIN + (waiting_this - waiting_cross/2), T_GREEN_MIN, T_GREEN_MAX)
```

Heavy demand on the active direction extends the green; demand on the conflicting direction shortens it. The result is bounded between the configured minimum and maximum, so real-time guarantees are preserved.

4.4 Real-time deadlines verified

- **Yellow duration:** exactly `T_YELLOW` seconds — enforced by a counted sleep loop in the controller.

- **Pedestrian max wait:** if a pedestrian has been waiting longer than $T_PEDESTRIAN_MAX_WAIT$, the active green is cut short (but never below T_GREEN_MIN).
- **Emergency response:** upon detecting an emergency, the controller switches active greens to yellow within one iteration of its loop, then to red after T_YELLOW seconds, then opens the emergency direction — total response is $T_YELLOW + T_ALL_RED$.

5. Configuration File

The project specification requires that no numerical parameter be hard-coded — all tunable values must be read from a text file supplied on the command line. We chose a simple `KEY = value` syntax with `#` comments because it is human-readable, trivially editable, and can be parsed in < 30 lines of C.

The parser lives in `config.h`. It first calls `load_defaults()` so the system still runs if a key is missing, then overlays whatever the file provides. The file name is passed as `argv[1]` to `main`, so a different configuration can be tried just by running:

```
./main config/heavy_traffic.cfg
```

Excerpt from `config/system.cfg`:

```
T_GREEN_MIN      = 8
T_GREEN_MAX      = 15
T_YELLOW         = 3
T_ALL_RED        = 2
T_PEDESTRIAN     = 10
VEHICLE_SPAWN_RATE = 25
```

6. OpenGL Visualisation

We chose `freeglut` over `GLFW` or `SDL` because `freeglut` is the de-facto tool used in the Birzeit graphics curriculum and ships in the `freeglut3-dev` Ubuntu package — no extra dependencies for the grader to install. The renderer is a pure consumer of the shared-memory blackboard: every frame it takes a lock, copies the blackboard into a local snapshot, releases the lock and then renders without holding it. This minimises lock-hold time to a few microseconds and keeps the real-time loops responsive.

6.1 Scene

- A green grass plane with gray asphalt roads, dashed yellow lane dividers, stop lines and zebra crossings drawn with `GL_QUADS`.
- Four 3-D traffic-light poles at the intersection corners; the lamp matching the current colour is made *emissive* so it visibly glows.
- Cars are 3-D boxes with darker cabins and four spherical wheels; emergency vehicles get a flashing red/blue lightbar.
- Pedestrians are simple stick figures (body box, head sphere, two legs); they animate across the zebra crossings during the pedestrian phase.
- A translucent HUD shows the current phase, countdown, light colours, queue lengths, pending pedestrians, and emergency status.

6.2 Why we did not use GLUT primitives only

The professor accepts "simple and elegant" elements; we went one step further and modelled real-looking cars, traffic-light housings, and stick-figure pedestrians using composed primitives because the rendering is the only feedback the grader sees. A first-look that shows a credible intersection is worth more than any number of command-line printouts.

7. Safety, Faults and Monitoring

- The function `state_is_safe()` is called after every transition; if it returns 0 the controller forces every light to red and increments the `safety_violations` counter.
- Each traffic-light process has a 0.2% per-command chance of simulating a hardware fault. The fault is reported as an `MTYPE_EVT_FAULT` event, and the lamp "fails safe" to red, never to green or yellow.
- The logger writes every event with severity (INFO / WARN / ERROR), timestamp, and source process, both to `logs/system.log` and to `stdout`.
- `main` traps `SIGINT` and `SIGTERM`, forwards them to its children, and removes every IPC object so the kernel is left clean.

8. Build and Test

Build: `make (release) or make debug (for gdb)`.

Run with GUI: `./main config/system.cfg`

Run without GUI: `./main config/system.cfg --no-gui`

Clean lingering IPC after a crash: `make ipcclean`

8.1 Manual test plan

- **Sanity:** launch and observe NS-GREEN → NS-YELLOW → ALL-RED → EW-GREEN → ... cycle.
- **Vehicle injection:** press *n/s/e/w* in the GUI window and watch the queue counts in the HUD.
- **Pedestrian:** press *N/S/E/W*; verify that the pedestrian phase appears between the next ALL-RED, never while any direction is green.
- **Emergency:** press *1/2/3/4*; verify that the active green first goes through yellow and red before the emergency direction turns green.
- **Crash recovery:** `kill -9` the controller while running; observe that the `SEM_UNDO` flag releases the mutex automatically; `main` detects the dead child and shuts the system down cleanly.

8.2 Verified properties

- The system was run for > 30 minutes under the default configuration; the safety-violation counter stayed at 0.
- Compiles cleanly with `gcc -Wall -Wextra -O2`, no warnings.
- All IPC objects are removed after every successful exit (verified with `ipcs`).

9. Conclusion

The chosen architecture — a semaphore-protected shared-memory blackboard for state, message queues for events, and one dedicated process per logical responsibility — produces a clean, modular, real-time system that satisfies every requirement of the project specification. Each IPC mechanism is used where it does the most good and is avoided where it would add latency or complexity. The configuration file removes hard-coded values, the OpenGL GUI makes the

simulation immediately understandable, and the logger gives the grader a precise audit trail of every event.

— *End of design report* —